S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 04

Aim: Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behavior by creating orphan and zombie processes.

Name: Sonu Shingare

USN: CD24053

Semester / Year: 4th SEM / 2nd Year

Academic Session: 2025-26

Date of Performance:

Date of Submission:

❖ **Aim:** Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behaviour by creating orphan and zombie processes.

❖ **Tasks to be done in this Practical.**

1. Adamis working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?
   - Kill processes by name
   - Kill a process based on the process name
   - Kill a single process at a time with the given process ID

2. Write a program for process creation using C
   - Orphan Process
   - Zombie Process

3. Create the process using fork () system call.
   - Child Process creation
   - Parent process creation
   - PPID and PID

❖ **Objectives:**

a. Implement process creation using the fork() system call to generate parent and child processes.

b. Retrieve and display the Process ID (PID) and Parent Process ID (PPID) for both processes.

c. Analyse system behaviour by creating and observing orphan and zombie processes.

❖ **Requirements:**

√ **Hardware Requirements:**

- Processor: Minimum 1 GHz.
- RAM: 512 MB or higher.
- Storage: 100 MB free space.

√ **Software Requirements:**

- Operating System: Linux/Unix-based.
- Compiler: GCC Compiler.
- Text Editor: Nano, Vim, or any preferred editor.

❖ Theory:

In many operating systems, the fork system call is an essential operation. The fork system call allows the creation of a new process. When a process calls the fork(), it duplicates itself, resulting in two processes running at the same time. The new process that is created is called a child process. It is a copy of the parent process. The fork system call is required for process creation and enables many important features such as parallel processing, multitasking, and the creation of complex process hierarchies. It develops an entirely new process with a distinct execution setting. The new process has its own address space, and memory, and is a perfect duplicate of the caller process.

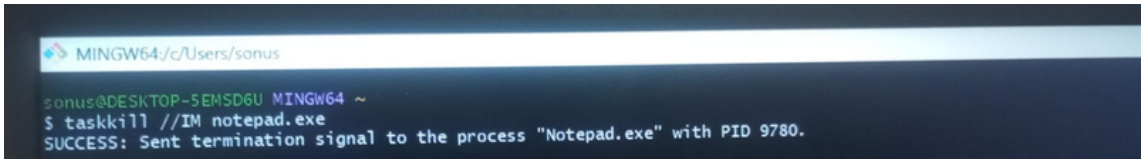
Basic Terminologies Used in Fork System Call in Operating System

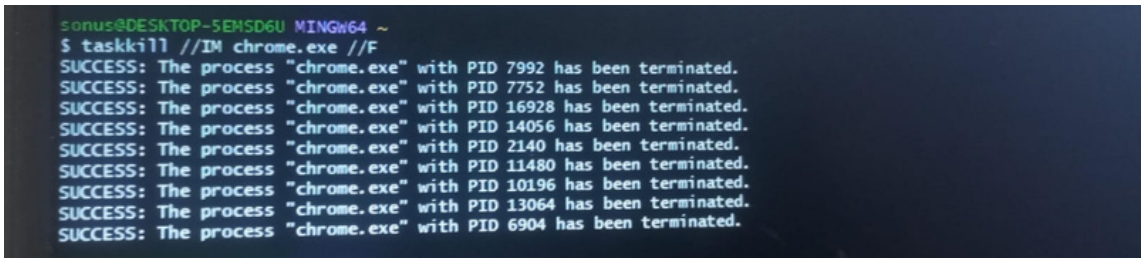
- ❖ **Process:** In operating system, a process is an instance of a program that iscurrently running. It is a separate entity with its own memory, resources, CPU, I/O hardware, and files.
- ❖ **Parent Process:** The process that uses the fork system call to start a new child process is referred to as the parent process. It acts as the parent process's beginning point and can go on running after the fork.
- ❖ **Child Process:** The newly generated process as a consequence of the fork system call is referred to as the child process. It has its own distinct process ID (PID), and memory, and is a duplicate of the parent process.
- ❖ **Process ID:** A process ID (PID) is a special identification that the
- ❖ operating system assigns to each process.
  Copy-on-Write: The fork system call makes use of the memory management strategy known as copy-on-write. Until one of them makes changes to the shared memory, it enables the parent and child processes to share the same physical memory. To preserve data integrity, a second copy is then made.
- ❖ **Return Value:** The fork system call's return value gives both the parent and child process information. It assists in handling mistakes during process formation and determining the execution route.

Process Creation

When the fork system call is used, the operating system completely copies the parent process to produce a new child process. The memory, open file descriptors, and other pertinent properties of the parent process are passed down to the child process. The child process, however, has a unique execution route and PID. The copy-on-write method is used by the fork system call to maximize memory use. At first, the physical memory pages used by the parent and child processes are the same. To avoid unintentional changes, a separate copy is made whenever either process alters a shared memory page.

The return value of the fork call can be used by the parent to determine the execution path of the child process. If it returns 0 then it is executing the child process, if it returns -1 then there is some error; and if it returns some positive value, then it is the PID of the child process.





Advantagesof ForkSystemCall

- **Creatingnew processes** with the fork system call facilitates therunning of several tasks concurrently within an operating system. The system'sefficiency and multitasking skills are improved by this concurrency. **Code reuse:** The child
- process inherits an exact duplicate of the parent process, including every code segment, when the fork system call is used. By using existing code, this feature encourages code reuse and streamlines the creation of complicated programmes. **Memory Optimisation:** When using the fork system call, the copy-on-write
- method optimises the use of memory. Initial memory overhead is minimised since parent and child processes share the same physical memory pages. Only when a process changes a shared memory page, improving memory efficiency, does copying take place. Process isolation is achieved by giving each process started by the fork system call its own memory area and set of resources. System stability
- and security are improved because of this isolation, which prevents processes interfering with one another. Disadvantages of Fork System Call

Memory Overhead: The fork system call has memory overhead even with the from copy-on-write optimisation. The parent process is first copied in its entirety, including all of its memory, which increases memory use.

- **Duplication of Resources:** When a process forks, the child process duplicates all open file descriptors, network connections, and other resources. This duplication may waste resources and perhaps result in inefficiencies.
- **Communication complexity:** The fork system call generates independent processes that can need to coordinate and communicate with one another. To enable data transmission across processes, interprocess communication
-

methods, such as pipes or shared memory, must be built, which might add complexity.

- **Impact on System Performance:** Forking a process duplicates memory allocation, resource management, and other system tasks. Performance of the system may be affected by this, particularly in situations where processes are often started and stopped.

❖ CODES

1. FORK.c :

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano fork.c
```

```
GNU nano 8.7                                   fork.c
#include <stdio.h>
#include <windows.h>

int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    printf("Parent Process Started!\n");

    // Create child process (opens cmd)
    if (!CreateProcess(
            NULL,
            "cmd.exe",
            NULL,
            NULL,
            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi
        )) {
        printf("CreateProcess failed. Error = %lu\n", GetLastError());
        return 1;
    }

    printf("Child process created successfully!\n");
    printf("Child PID: %lu\n", pi.dwProcessId);

    // Wait for child to finish
    WaitForSingleObject(pi.hProcess, INFINITE);

    // Close handles
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    printf("Child process finished.\n");

    return 0;
}
^G Help   ^O Write Out   ^W Where Is   ^K Cut   ^T Execute   ^C Location   M-U Undo
^X Exit   ^R Read File   ^\ Replace    ^U Paste ^J Justify   ^/ Go To Line M-E Redo
```

Output :

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ gcc fork.c -o fork && ./fork
Parent Process Started!
Child process created successfully!
Child PID: 8072
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.
```

2. Orphan.c:

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano pro.c
```

```
GNU nano 8.7                                    pro.c
#include <stdio.h>

int main() {
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;

    printf("Sum = %d\n", sum);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line
M-U Undo
M-E Redo
```

Output:

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ gcc pro.c -o pro && ./pro
Enter two numbers: 4
2
Sum = 6
```

3. Zombie.c:

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano zom.c
```

```
GNU nano 8.7                                   zom.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child process finished\n");
    }
    else {
        // Parent process
        printf("Parent process sleeping...\n");
        sleep(20);   // Parent sleeps, child becomes zombie
        printf("Parent process exiting\n");
    }

    return 0;
}

^G Help   ^O Write Out   ^W Where Is   ^K Cut   ^T Execute   ^C Location
^X Exit   ^R Read File   ^\ Replace    ^U Paste ^J Justify   ^/ Go To Line
```

Output:

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano zom.c

sonus@DESKTOP-5EMSD6U MINGW64 ~
$ gcc zom.c -o zom &&./zom
Enter a positive number: 6
Numbers from 1 to 6:
1 2 3 4 5 6
```

❖ **Conclusion**: In this practical, we conclude that Linux commands like pkill, killall, and kill effectively reduce system load by terminating processes based on their name or ID, improving system performance and stability.

❖ Discussion Questions:

1. What is the concept of pointers in C?

Ans: A pointer in C is a variable that stores the memory address of another variable. It enables direct access tomemory, facilitating dynamic memory management and efficient function calls, especially for large data structures.

2. What is the difference between malloc and calloc in C?

Ans: malloc allocates uninitialized memory, while calloc allocates memory and initializes all bits to zero. Use malloc when initialization is not required and calloc when zeroed memory is needed.

3. What is the purpose of the fork() system call in Unix?

Ans: fork() creates a new process by duplicating the parent process. It allows for parallel execution of processes, where the child process gets a copy of the parent's address space.

4. What is the difference between struct and union in C?

Ans: A struct allocates separate memory for each member, while a union shares the same memory space for all members, saving memory but allowing only one member to hold a value at a time.

5. What does grep do in Linux?

Ans: grep searches for specific patterns in files and outputs the lines containing those patterns. It's a powerful tool for text processing and log analysis in Linux environments.

❖ References:

https://www.geeksforgeeks.org/fork-system-call-in-operating-system/?ref=ml_lbp
https://www.scaler.com/topics/fork-system-call/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(DS)
Sem: 4 / 2025-26